$SiER^2NA$

Simple Extensible Register-Register Neumann Architecture

Instruction Set Architecture

1.0

October 12, 2026

Contents

1 Preface **2**

2 Glossary **2**

3 Registers **2**

3.1 Pointer registers . . . 3
3.1.1 rfp - frame pointer . . . 3
3.1.2 rrap - interrupt routine array pointer . . . 3
3.1.3 rlfp - last frame pointer . . . 3
3.1.4 rip - instruction pointer . . . 3
3.1.5 rtp - procedure tail pointer . . . 3
3.1.6 rsp - stack pointer . . . 3
3.2 Restricted purpose . . . 3
3.2.1 rz - zero register . . . 3
3.3 General purpose . . . 3
3.3.1 r0 . . . 3
3.3.2 r1 . . . 4
3.3.3 r2 . . . 4
3.3.4 r3 . . . 4
3.3.5 r4 . . . 4
3.3.6 r5 . . . 4
3.3.7 r6 . . . 4
3.3.8 r7 . . . 4
3.3.9 r8 . . . 4
3.3.10 r9 . . . 4
3.3.11 r10 . . . 4
3.3.12 r11 . . . 4
3.3.13 r12 . . . 4
3.3.14 r13 . . . 4
3.3.15 r14 . . . 5
3.3.16 r15 . . . 5

4 Interrupts **5**
4.1 Interrupt List 6
4.2 Interrupt Routine Array 6
4.2.1 Interrupt Routine Array Entry 6
4.3 Interrupt Routine Handler 6
4.4 User-defined interrupts 7

5 Structures **7**
5.1 CPUID 7

1 Preface

2 Glossary

- Word - 32-bit value
- Half-word - 16-bit value
- Byte - 8-bit value

3 Registers

The SyrENA architecture, similarily to the TMS9900, does not store general-purpose registers on the CPU, but rather a *frame pointer* that points to the address in register RAM of the current *register frame* for the executing context.

A *register frame* refers to a (virtual) structure in RAM of 128 bytes (1024 bits) in length that stores 32 4-byte values total:

Table 1: Register frame (1023-768)

1023..992	991..960	959..928	927..896	895..864	863..832	831..800	799..768
reserved							

Table 2: Register frame (767-512)

767..736	735..704	703..672	671..640	639..608	607..576	575..544	543..512
reserved				rlfp	rsp	rtp	rip

Table 3: Register frame (511-256)

511..480	479..448	447..416	415..384	383..352	351..320	319.288	287..256
r15	r14	r13	r12	r11	r10	r9	r8

Table 4: Register frame (255-0)

255..224	223..192	191..160	159..128	127..96	95..64	63..32	31..0
r7	r6	r5	r4	r3	r2	r1	r0

Registers are categorised into two categories:

- **Global** registers are stored directly on the CPU and their values persist between frame changes
- **Local** registers are stored in the *register frame* and their address and/or value depends on the current *frame pointer*

3.1 Pointer registers

3.1.1 rfp - frame pointer

Availability: Global
Stores the address to the current *register frame*

3.1.2 rrap - interrupt routine array pointer

Availability: Global
Stores the address to the current Interrupt Routine Array. Set by the system to indicate which IRA the CPU should use for interrupt handling

3.1.3 rlfp - last frame pointer

Availability: Local
Stores the address to the previous *register frame*. Used when returning from interrupt routines

3.1.4 rip - instruction pointer

Availability: Local
Stores the address of the currently executing instruction

3.1.5 rtp - procedure tail pointer

Availability: Local
Stores the address of the instruction that called the current procedure

3.1.6 rsp - stack pointer

Availability: Local
Stores the address of the current stack frame

3.2 Restricted purpose

3.2.1 rz - zero register

Availability: Global
Always 0. Writes are ignored

3.3 General purpose

3.3.1 r0

Availability: Local

3.3.2 r1

Availability: Local

3.3.3 r2

Availability: Local

3.3.4 r3

Availability: Local

3.3.5 r4

Availability: Local

3.3.6 r5

Availability: Local

3.3.7 r6

Availability: Local

3.3.8 r7

Availability: Local

3.3.9 r8

Availability: Local

3.3.10 r9

Availability: Local

3.3.11 r10

Availability: Local

3.3.12 r11

Availability: Local

3.3.13 r12

Availability: Local

3.3.14 r13

Availability: Local

3.3.15 r14

Availability: Local

3.3.16 r15

Availability: Local

4 Interrupts

Interrupts are handled by the CPU after every execute cycle of a single instruction. The CPU checks if any interrupts were raised after executing the instruction and, if so, checks and picks the correct Interrupt Routine Handler from the current Interrupt Routine Array to handle the raised interrupt.

If the `enable` bit of the IRA entry is not set, the interrupt is not handled in software, but the exact behaviour depends on the type of the raised interrupt.

There are three types of interrupts:

Type	SW	HW	Recoverable	Pauses execution
Signal	✓	✓	✓	✗
Trap	✓	✓	✓	✓
Fault	✗	✓	✗	✓

- *Signal* interrupts do not pause execution and do not have to be explicitly handled in any way. This type of interrupt is recommended for general hardware-software communication.
- Raised *trap* interrupts are most commonly recoverable errors and will pause execution of the frame that the trap occurred in if no valid IRH was found to handle the raised interrupt.
- Raised *fault* interrupts are non-recoverable and reset the CPU upon existing from the IRH or if no valid handler was found. These cannot be raised manually from software.

Upon startup the *power-on program* must set the `rrap` register to a new empty, zero-filled IRA. Upon taking control, the operating system should allocate its own IRA, filling it with whatever entries are necessary to handle wanted interrupts, and set the value of `rrap` to the address of said IRA.

4.1 Interrupt List

ID	Mnemonic	Name	Type	Notes
0x00	#UDO	Undefined Opcode	Trap	
0x01	#DBG	Debug	Trap	
0x02	#BRK	Breakpoint	Trap	
0x03	#IOP	Invalid Operation	Trap	...
0x04	#DBZ	Division By Zero	Trap	
0x05 .. 0x1F				reserved
0x20 .. 0xFF				user-defined

4.2 Interrupt Routine Array

Offset	Value
0x00	Entry for INT 0x00
0x0A	Entry for INT 0x01
0x14	Entry for INT 0x02
...	...
0x9F6	Entry for INT 0xFF
0xA00	

4.2.1 Interrupt Routine Array Entry

79..48	47..16	15..1	0
&rp	&rfp	–	en
Interrupt Routine Handler	Register Frame	Reserved	Enabled

4.3 Interrupt Routine Handler

Interrupt Routine Handlers (IRH) are user-defined procedures that respond to a specific interrupt raised by the CPU. IRs have their own *register frame* and can access the previous *register frame* via its address located in the `rlfp` register.

IRHs can exit at any time and resume execution by setting the current *register frame* to the previous *register frame*: `mov rfp, rlfp`. For non-recoverable interrupts this will result in the CPU being reset.

4.4 User-defined interrupts

5 Structures

5.1 CPUID

